

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce Department of Information Technology (B.Sc.I.T Semester V)

ARTIFICIAL INTELLIGENCE (AI)

Practical – II

Roll No.: T010	Name: Sagar Kandari
Class: TYIT	Batch:1
Date of Assignment: 11/07/2026	Date/Time of Submission: 11/07/2026

Problem Solving by Searching (Uninformed Search)

2. Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search (BFS).

Code:

```
from collections import deque

# Function to display puzzle state
def print_state(state):
    for i in range(0, 9, 3):
        print(state[i], state[i + 1], state[i + 2])
    print()

# Function to find possible next states
def get_neighbors(state):
    neighbors = []

    # Find position of blank tile
    blank_index = state.index(0)

    # Possible moves: Up, Down, Left, Right
    moves = {
        "Up": -3,
        "Down": 3,
        "Left": -1,
        "Right": 1
    }

    for move, position_change in moves.items():
        new_index = blank_index + position_change

        # Check valid moves
        if move == "Up" and blank_index < 3:
            continue
```

✓ 0.0s

```

        if move == "Down" and blank_index > 5:
            continue
        if move == "Left" and blank_index % 3 == 0:
            continue
        if move == "Right" and blank_index % 3 == 2:
            continue

        # Create new state by swapping blank tile
        new_state = list(state)
        new_state[blank_index], new_state[new_index] = (
            new_state[new_index],
            new_state[blank_index]
        )

        neighbors.append((tuple(new_state), move))

    return neighbors

# BFS function
def bfs(initial_state, goal_state):
    queue = deque()

    # Each queue item contains: current_state, path
    queue.append((initial_state, []))

    visited = set()
    visited.add(initial_state)

    while queue:
        current_state, path = queue.popleft()

```

```

        # Check if goal is reached
        if current_state == goal_state:
            return path

        # Generate next possible states
        for neighbor, move in get_neighbors(current_state):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [(move, neighbor)]))

    return None

# Main Program
initial_state = (
    1, 2, 3,
    4, 0, 5,
    6, 7, 8
)

goal_state = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

print("Initial State:")
print_state(initial_state)

print("Goal State:")
print_state(goal_state)

```

0.0s

```

solution = bfs(initial_state, goal_state)

if solution:
    print("Shortest sequence of moves:")
    current_step = 1

    for move, state in solution:
        print("Step", current_step, "- Move:", move)
        print_state(state)
        current_step += 1

    print("Goal reached successfully.")
    print("Total number of moves:", len(solution))
else:
    print("No solution found.")

print("Sagar Kandari T010.")

```

✓ 0.0s

Output:

```

... Initial State:
1 2 3
4 0 5
6 7 8

Goal State:
1 2 3
4 5 6
7 8 0

Shortest sequence of moves:
Step 1 - Move: Right
1 2 3
4 5 0
6 7 8

Step 2 - Move: Down
1 2 3
4 5 8
6 7 0

Step 3 - Move: Left
1 2 3
4 5 8
6 0 7
...

Goal reached successfully.
Total number of moves: 14
Sagar Kandari T010.

```